TYPESCRIPT DECORATORS + NESTJS
Від «магічного тега» до власного IoC-контейнера

Заняття №6

Node.js Pro

Лектор
Кирило Орлов

r_d

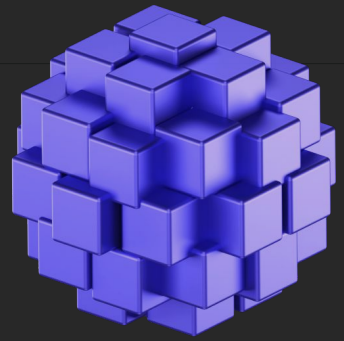
ПРОБЛЕМА: «МАГІЧНІ ТЕГИ», ЯКІ НІХТО НЕ ЧИТАЄ

@Injectable() над класом — і воно якось працює

@Inject(TOKEN) у конструкторі — звідки Nest знає тип?

забув один прапорець у tsconfig — падіння в РАНТАЙМІ

помилка «Nest can't resolve dependencies» нічого не пояснює



Усе це — один механізм: компілятор кладе типи в метадані, контейнер їх читає. Розберемо його до дна, і повідомлення про помилку стане зрозумілим.

СТЕК НА СЬОГОДНІ: DECORATORS · REFLECT-METADATA · NESTJS DI

Перша частина — декоратори (45 хв)	Друга частина — NestJS (40 хв)
чотири види: клас, метод, проперті, параметр	модулі — межа видимості провайдерів
decorator factory — декоратор із аргументами	провайдери й токени — те саме, що ми напишемо руками
reflect-metadata — де живуть метадані	scopes і lifecycle hooks
design:paramtypes — типи конструктора в рантаймі	Декоратор — це функція. Більше нічого

НАСКРІЗНА НИТКА
На L#4 ми вже написали контейнер — крок 8, сорок рядків: рекурсія, синглтони, детекція циклу. Сьогодні ми його НЕ переписуємо, а забираємо з нього одну річ: ручний список залежностей. Один граф, три стадії — L#4 руками → наш resolve → NestJS.

ДЕКОРАТОР — ЦЕ ФУНКЦІЯ. ОСЬ ДОКАЗ

```
// stages/01 — одна функція, два способи застосувати
function sealed(ctor: Function): void {
  console.log(`  sealed() отримав конструктор: ${ctor.name}`);
  Object.seal(ctor);
  Object.seal(ctor.prototype);
}

@sealed                       // TS-синтаксис
class WithAt { hello() { return 'hi'; } }

class WithoutAt { hello() { return 'hi'; } }
sealed(WithoutAt);            // той самий виклик руками

// $ npm run step1
//   sealed() отримав конструктор: WithAt
//   sealed() отримав конструктор: WithoutAt
// Object.isSealed:
//   WithAt    → true
//   WithoutAt → true
//   спроба додати метод у WithAt: TypeError
// ⚠️ омонім із L#4: app.decorate() у Fastify (крок 9) — не @,
//   а «допиши властивість в об'єкт». Тут декоратор = @-функція
```

@ — синтаксичний цукор над викликом функції (L#4)

1 · @sealed = sealed(WithAt): компілятор кличе функцію

2 · sealed(WithoutAt) руками — лог і ефект однакові

3 · @f() — фабрика: повертає декоратор (крок 3)

викликається ОДИН раз при оголошенні класу

урок: усе, що вміє декоратор, ти можеш зробити вручну

ЧОТИРИ ВИДИ ДЕКОРАТОРІВ І ЩО КОЖЕН ОТРИМУЄ

01. Клас

(target) — сам конструктор. Може повернути новий клас

02. Метод

(target, key, descriptor) — можна підмінити реалізацію

03. Властивість

(target, key) — descriptor немає, лише позначити

04. Параметр

(target, key, index) — тільки індекс. Значення НЕ бачить

05. Порядок

члени — в порядку оголошення; клас — завжди останній

06. Коли

під час ОГОЛОШЕННЯ класу, не під час виклику

СИГНАТУРИ: ЩО САМЕ ОТРИМУЄ КОЖЕН ДЕКОРАТОР

Вид	Аргументи	Що можна зробити
Клас	(constructor)	записати метадані, підмінити клас цілком
Метод	(target, key, descriptor)	обгорнути виклик, змінити реалізацію
Властивість	(target, key)	лише позначити — дескриптора немає
Параметр	(target, key, index)	запам’ятати індекс — значення побачить хтось інший
Фабрика	(...args) => decorator	передати аргумент: @Roles('admin')

ПОРЯДОК І КОМПОЗИЦІЯ: ЩО ЗА ЧИМ ВИКОНУЄТЬСЯ

фабрики обчислюються ЗГОРИ ВНИЗ

самі декоратори застосовуються ЗНИЗУ ВГОРУ

члени класу — в порядку оголошення, не за видом

@A @B foo() → B(foo), потім A(результат)

декоратор класу — №18, останній; його фабрика — №15

усе це — під час ОГОЛОШЕННЯ, до створення першого екземпляра

```
$ npm run step3 — повний клас, 18 кроків:

 1    фабрика   PropA      ← властивість перша,
 2    декоратор PropA        бо оголошена першою
 3    фабрика   MethodOne
 4-5  фабрики   P-one-0, P-one-1   зліва направо
 6-7  декоратори P-one-1, P-one-0  справа наліво
 8    декоратор MethodOne  ← параметри ПЕРЕД ним
 9-14 PropB, MethodTwo — за тим самим правилом
15    фабрика   Class      ← ще ДО параметрів
ctor
16-17 фабрика+декоратор P-ctor-0
18    декоратор Class      ← клас завжди останній
```



СТАДІЯ 1: РУКАМИ

```
// стадія 1 — повторення з L#4 (крок 8)
// граф, який житиме до кінця лекції:
// UserService ← UserRepo ← Logger ← Config

const config  = new Config();
const logger  = new Logger(config);
const repo    = new UserRepo(logger);
const service = new UserService(repo, logger);

// додав залежність у конструктор —
// правиш КОЖНЕ місце створення і всі тести.
// На L#4 цю роботу вже забрав контейнер.
// Новина L#6 — він читає типи CAM.
```

- Один клас пройде всю лекцію наскрізь
- стадія 1 · руками — це вже було на L#4 (крок 8): повторюємо за 30 секунд
- стадія 2 · міні-контейнер: container.resolve(UserService) — сьогодні починаємо звідси
- стадія 3 · NestJS: providers: [UserService, ...]

«НАЛІПКА НА КОРОБЦІ»: ДЕКОРАТОР ЯК АНАЛОГІЯ

ЧАСТИНА TS

- клас
- @Injectable()
- метадані (reflect-metadata)
- контейнер, що читає метадані
- декоратор без reflect-metadata

АНАЛОГІЯ (СКЛАД)

- коробка на складі
- наліпка «крихке»
- журнал складу: що на якій коробці
- комірник, який діє за журналом
- наліпка є, журналу немає — ніхто її не побачить

Де аналогія ламається: наліпку клеїть людина, а метадані пише компілятор — і тільки якщо ти попросив прапорцем emitDecoratorMetadata. Забув прапорець — коробка позначена, а журнал порожній.





```
// tsconfig.json (з репо, скорочено) — без цих двох прапорців нічого не працює
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "strict": true,
    "experimentalDecorators": true,   // ← legacy-модель (NestJS 11)
    "emitDecoratorMetadata": true     // ← без нього немає design:paramtypes
  }
}

// main.ts — НАЙПЕРШИМ рядком, один раз на застосунок
import 'reflect-metadata';

// stages/04 — звідки в рантаймі типи конструктора:
@Injectable()
class UserService {
  constructor(log: Logger, cfg: Config, port: number, name: string, mail: Mailer) {}
}
Reflect.getMetadata('design:paramtypes', UserService);
// → [Logger, Config, Number, String, Object]  ← конструктори, не рядки; Mailer став Object

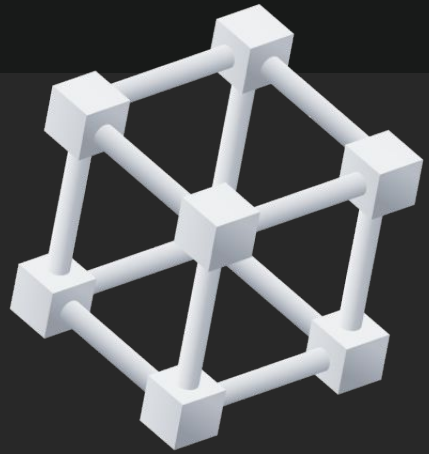
// ⚠️ tsx (esbuild) метадані НЕ емітить: npm run step4:tsx → undefined
//    тому кроки 4–6 на ts-node; курс на TypeScript 6.x (у репо — 6.0.3)
```

DESIGN:PARAMTYPES: ЗВІДКИ КОНТЕЙНЕР ЗНАЄ ТИПИ

компілятор емітить типи параметрів у метадані класу

але ТІЛЬКИ якщо на класі є хоч один декоратор

інтерфейс у метаданих стає Object — типів там уже немає



Саме тому для інтерфейсів потрібен явний @Inject(TOKEN): типи стираються при компіляції, а токен — це значення, яке доживає до рантайму. Клас доживає теж — він і є свій власний токен.

ЧОМУ ВОНО КОМПІЛЮЄТЬСЯ, А ПАДАЄ В РАНТАЙМІ

reflect-metadata не імпортовано
падає перший виклик: TypeError: Reflect.getMetadata is not a function

emitDecoratorMetadata вимкнено
метадані порожні → deps = [] → конструктор без аргументів

На класі немає декоратора
компілятор НЕ емітить метадані взагалі — навіть із прапорцем

Найпідступніший випадок
Клас без жодного декоратора не отримує design:paramtypes, навіть коли emitDecoratorMetadata увімкнено. Компілятор емітить метадані лише тоді, коли на класі вже є хоч один декоратор. Тому @Injectable() на класі без залежностей — не формальність: без нього наступний клас, що його інжектить, отримає undefined.

«ЗАМОВЛЕННЯ НА СКЛАДІ»: IoC-КОНТЕЙНЕР

ЧАСТИНА NEST

- провайдер
- токен
- module.providers
- module.exports
- moduleRef.get(Token)

АНАЛОГІЯ (СКЛАД)

- товар, який комірник уміє видати
- артикул
- каталог цього складу
- що склад віддає сусіднім складам
- заявка: «дайте артикул X»



```
// stages/05 — той самий контейнер, що на L#4 (крок 8), але типи бере CAM
// списку deps немає ніде · клас Container: 40 рядків → 19 · ядро ДЗ #6
const INJECTABLE = Symbol('injectable');
const Injectable = (): ClassDecorator => (t) => Reflect.defineMetadata(INJECTABLE, true, t);
class Container {
  private singletons = new Map<Ctor, unknown>();
  resolve<T>(target: Ctor<T>, path: string[] = []): T {
    if (this.singletons.has(target)) return this.singletons.get(target) as T;
    if (path.includes(target.name))
      throw new Error(`цикл залежностей: ${[...path, target.name].join(' -> ')}`);
    if (!Reflect.getMetadata(INJECTABLE, target))
      throw new Error(`${target.name} не позначений @Injectable()`);
    const deps = (Reflect.getMetadata('design:paramtypes', target) ?? []) as Ctor[];
    console.log(`  resolve ${target.name} ← [${deps.map((d) => d.name).join(', ')}]`);
    const args = deps.map((d) => this.resolve(d, [...path, target.name]));
    const instance = new target(...args);      // граф зібрано рекурсивно
    this.singletons.set(target, instance); return instance;
  }
}
// $ npm run step5 → resolve UserService ← [UserRepo, Logger] · UserRepo ← [Logger]
//   Logger ← [Config] · Config ← [] · результат: { id: '42' } · Синглтон: true
```

```
// стадія 3: той самий граф зі stages/05 — тепер декларативно (NestJS 11)
import { Module, Injectable, Inject } from '@nestjs/common';
import { ConfigService } from './config.module';   // dynamic module — далі
export const CONFIG = Symbol('CONFIG');            // токен для не-класу
@Injectable() export class Config { readonly dbUrl = 'postgres://localhost/demo'; }
@Injectable() export class Logger { constructor(private cfg: Config) {} }
@Injectable() export class UserRepo { constructor(private logger: Logger) {} }
@Injectable()
export class UserService {
  constructor(private repo: UserRepo, private logger: Logger,
              @Inject(CONFIG) private cfg: { url?: string },   // тип або токен
              private config: ConfigService) {}    // із ConfigModule.forRoot
}
@Module({
  providers: [Config, Logger, UserRepo, UserService,
              { provide: CONFIG, useValue: { url: process.env.DATABASE_URL } }],
  exports: [UserService],                          // видно іншим модулям
})
export class UserModule {}
// NestFactory.create() збере граф так само, як наш resolve() зі stages/05
```



МОДУЛІ: МЕЖА ВИДИМОСТІ, А НЕ ТЕКА

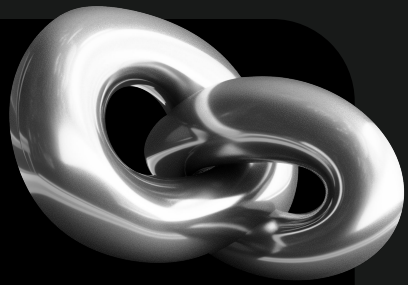
01. providers — що модуль уміє створювати

02. exports — що з цього бачать інші модулі

03. imports — чиї exports я хочу бачити

04. не експортував — провайдер живе тільки всередині

05. @Global() — видно всім без imports. Використовувати рідко



```
// Dynamic module: патерн forRoot, який ти бачитимеш скрізь
import { Module, DynamicModule } from '@nestjs/common';

export const CONFIG_OPTIONS = 'CONFIG_OPTIONS';   // токен-рядок

@Module({})
export class ConfigModule {
  static forRoot(opts: { path: string }): DynamicModule {
    return {
      module: ConfigModule,
      providers: [
        { provide: CONFIG_OPTIONS, useValue: opts },
        ConfigService,
      ],
      exports: [ConfigService],
      global: true,
    };
  }
}

// у AppModule:
@Module({ imports: [ConfigModule.forRoot({ path: '.env' })] })
export class AppModule {}

// forRootAsync — те саме, але useFactory з інжекцією залежностей
```

ЖИТТЄВИЙ ЦИКЛ NESTJS-ЗАСТОСУНКУ

```
  СТАРТ
    ↓
NestFactory.create(AppModule)
    ↓
  сканування модулів     imports → providers → controllers
    ↓
  створення провайдерів   граф залежностей, рекурсивно
    ↓
  onModuleInit            модуль готовий · підключення до БД
    ↓
  onApplicationBootstrap  усі модулі готові
    ↓
  app.listen()            сервер приймає запити
    ↓
═══════ ПРАЦЮЄ ═══════
    ↓
  SIGTERM
    ↓
  onModuleDestroy         зупиняємо таймери, воркери
    ↓
  beforeApplicationShutdown
    ↓
  onApplicationShutdown   закриваємо пули БД, черги
    ↓
  ПРОЦЕС ЗАВЕРШЕНО
```

⚠ enableShutdownHooks() — інакше останні три не викличуться



SCOPES I LIFECYCLE HOOKS

DEFAULT — синглтон. 99% випадків. ⚠ в Inversify (L#4) дефолт — transient

REQUEST — новий екземпляр на кожен запит. Дорого

TRANSIENT — новий на кожного, хто інжектить

⚠ REQUEST-scope піднімається вгору: усі, хто його інжектить, теж стають request-scoped

onModuleInit — тут підключення до БД, не в конструкторі

onApplicationShutdown — graceful: дозакрити пули й черги

SCOPES: ЩО ОБИРАТИ І ЧОГО ЦЕ КОШТУЄ

Scope	Коли створюється	Ціна
DEFAULT	один раз при старті застосунку	нуль
REQUEST	на кожен HTTP-запит	висока: граф перебудовується щоразу
TRANSIENT	на кожного споживача окремо	середня
Заражання	REQUEST тягне за собою всіх, хто його інжектить	легко зробити весь застосунок request-scoped
Альтернатива	AsyncLocalStorage замість REQUEST-scope	нуль — розберемо на L#8

```
// Циклічна залежність: A потребує B, B потребує A — граф нема з чого почати

// наш контейнер зі stages/05 падає зі зрозумілим ланцюгом (реальний вивід step5):
//   Error: цикл залежностей: A -> B -> A
// ⚠️ сама детекція — не новина: на L#4 (крок 8) вона вже була. Різниця в ланцюгу:
//   там токени, які написала людина; тут імена класів, знайдені компілятором
// 1) обхідний шлях у Nest — forwardRef: посилання стає лінивим (це пластир)
@Injectable()
export class AService {
  constructor(@Inject(forwardRef(() => BService)) private b: BService) {}
}

// 2) правильний шлях — розірвати цикл третьою сутністю:   A → Shared ← B
//    якщо A і B потрібні одне одному — між ними зазвичай
//    ховається третій сервіс, який ще не виділили

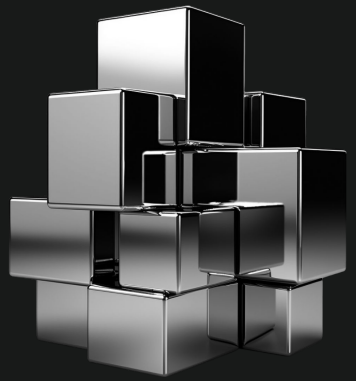
// у ДЗ #6 твій контейнер має так само назвати ВЕСЬ ланцюг:
//   AService -> BService -> AService
```

ЩО СВІДОМО ЛИШАЄМО НА ПОТІМ

Stage 3 декоратори (TC39) — інша модель: декоратор отримує (value, context). Стандарт майбутнього, але NestJS 11 тримається на legacy. Знати різницю треба, писати на них сьогодні — ні.

Контролери, DTO й параметр-декоратори — це L#7. Сьогодні тільки контейнер: як об’єкти створюються, а не як приходить HTTP-запит.

⚠ Повний lifecycle запиту (middleware → guard → interceptor → pipe → filter) — L#8. Це топ-1 питання минулого потоку, і воно заслуговує окремої лекції, а не абзацу.



ПІДСУМКИ + МІСТ ДО L#7

ДЕКОРАТОР

звичайна функція, яку компілятор викликає при оголошенні класу

МЕТАДАНІ

design:paramtypes існує лише з декоратором + emitDecoratorMetadata

NESTJS

той самий UserService — тепер декларативно; модуль — межа видимості

ДЗ #6

7 балів · власний IoC-контейнер: @Injectable, @Inject(token), резолв через design:paramtypes, scopes singleton/transient, детекція циклів. Тести обов’язкові. Частина 1 з 3 mini-Nest

ДАЛІ

L#7 — контролери, DTO й параметр-декоратори поверх цього ж механізму